手法の全体像

多端末の検証を1つにまとめて共有・検証する

― 何があって、何ができるのか ―

week19 / 01_手法の全体像

やりたいこと（前提）

各端末: RSSI検証＋距離検証＋複雑計算 を秘匿したまま証明 ← ZKP
複数端末を1つにまとめる（同じドローンにK台が整合） ← 集約の手法
みんなで共有・検証（ブロックチェーンに記録） ← 検証・共有の手法

■ 手法は大きく4つの層に分かれる

- 層1 = ゼロ知識証明で何ができるか（道具）
- 層2 = 証明系の種類（SNARK / STARK / Folding …）
- 層3 = 複数の証明をまとめる手法（本案件の肝）
- 層4 = 検証・共有の手法（オンチェーン / DA 層 …）

層1 — ゼロ知識証明(ZKP)で何ができるか

■ 「中身を見せずに、計算が正しいことだけを証明する」道具
■ 秘匿 = 座標・生RSSIを出さず「整合(1)/矛盾(0)」だけを示す
■ 正しさの保証 = 「30m以内かつ相対帯で整合」をごまかせない形で証明
■ 選択的開示 = drone_id・しきい値だけを公開、他は隠す
■ 集約の土台 = 「他の証明が正しい」ことすら証明できる（→層3の再帰）

■ ＝プライバシーを守ったまま、多端末の合意を検証可能にできる

層2 — 証明系の種類（性質の違うエンジン）

種類	証明サイズ	オンチェーン検証	事前準備	再帰	ひとこと
Groth16	最小	最安(≈348k)	回路ごと	可	小さく安い・準備が弱点
PLONK/Honk	小〜中	中〜高	1回	可	準備が緩い・汎用
STARK	大	高	不要	得意	準備不要・証明が大
Folding	最終のみ	圧縮依存	ほぼ不要	最得意	N個を逐次畳む・省メモリ

■ 現行の「回路別セレモニー(★1脆弱性)」は PLONK/STARK/Folding で構造的に消える

層3 — 複数の証明をまとめる手法（肝）

#	手法	まとめる場所	オンチェーン	現状
1	オンチェーン集計(K-of-N)	契約	台数分 O(N)	実装済 M1
2	回路内バッチ(1回路にN端末)	単一プローバ	1回・定数	試作
3	再帰的証明合成	集約器	1回・定数	実装済 M2
4	Folding(Nova系)	端末/コーディネータ	最終1回	未
5	専用証明集約(SnarkPack)	集約器	1回・対数	未
6	非ZKしきい値合意(閾値署名)	端末群	署名1個	未

■ 1(集計)と3(再帰)は実装済。未検討は4(Folding)/5/6

層4 — 検証・共有の手法

手法	できること	効く攻撃	現状
オンチェーンverifier	契約が証明を検証し記録	改ざん(暗号で不可)	実装済
オフチェーン検証＋コミット	重い検証は外で結果だけ記録	コスト削減	任意
DA層に提出集合コミット(網羅性)	全員分を含めた証明	検閲(間引き)	設計済 #3b
許可型チェーン(BFT合意+全複製)	参加者全員が同結果を保持	検閲・可用性	設計済

まとめ — 選ぶ順序

① ZKPで何を秘匿し何を示すか（層1）　… 済: 座標/RSSIを隠し「整合」だけ公開
② どの証明系エンジンか（層2）　… 現行Groth16 / ★1消すならPLONK系等
③ 多端末をどうまとめるか（層3）　… 済:1(集計)3(再帰) / 未:4(Folding)5,6
④ どう検証・共有するか（層4）　… 済: オンチェーン＋網羅性(#3b)

■ フレームワークは②〜③を実装する“乗り物”にすぎない

■ 未検討で価値ありは 手法4(Folding) ＝ 端末側で省メモリに集約

フレームワーク比較

何ができる / メリット / デメリット

早見表

フレームワーク	エンジン	事前準備	モバイル証明	一言
circom+rapidsnark	Groth16	回路別	○	現行の定番
gnark	Groth16/PLONK	回路別	×(重い)	集約(サーバ)
Noir+Barretenberg	Honk(PLONK系)	1回	◎	書きやすい・準備緩い
halo2	PLONK	1回/不要	○	強力だが難しい
Plonky2/3	STARK	不要	○	準備不要・再帰速い
Nova/Sonobe	Folding	ほぼ不要	◎(省メモリ)	端末側集約向き
RISC Zero/SP1	zkVM(STARK)	不要	△(重い)	普通のコードで書ける
Cairo/Stwo	STARK	不要	◎(実証多)	Starknet前提
Mopro	(上記を包む)	—	◎	証明系でなく橋渡し

① circom + rapidsnark（現行）

■ できること = circom記述→snarkjs/rapidsnarkでGroth16。circomlibにPoseidon/EdDSA/比較、実装済

■ メリット

- 証明が最小・オンチェーン検証が最安(≈348k gas)
- ライブラリと実績が豊富・既に動いている
- rapidsnarkでarm64実機証明が実用的

■ デメリット

- 回路ごとにトラステッドセットアップの儀式(★1脆弱性)
- 回路更新のたびに儀式のやり直し
- 低レベルで複雑な回路は書きにくい

■ 本案件 = 現状維持の基準。端末証明の土台として妥当

② gnark（現行の集約器）

■ できること = Goで回路記述、Groth16/PLONK。再帰検証で「N個の証明を検証する証明」(M2)

■ メリット

- プローバが高速・Goで実装しやすい
- 再帰・Merkle・MiMC が揃い集約に向く
- EVM向けverifierを出力できる

■ デメリット

- サーバ向き・端末での証明生成は重く非現実的
- Groth16なら回路別セットアップが必要

■ 本案件 = 集約(M2)はこれで継続。端末証明器の候補ではない

③ Noir + Barretenberg

■ できること = 高水準Noirで記述、bb(UltraHonk)。Poseidon標準・EdDSA実装あり・比較可→移植で実現可能

■ メリット

- 回路別セットアップ不要(1回使い回し)→★1を根治
- 書きやすく・クライアント証明が速い
- Mopro経由でiOS/Android/Flutterに載る

■ デメリット

- オンチェーン検証が約2.4M gas(Groth16の約7倍)
- circom→Noirの回路移植が必要
- 集約系を現行gnarkから組み替え

■ 本案件 = 「セットアップ根治＋端末で速く」の第一候補。gasと集約再設計がトレードオフ

④ halo2

■ できること = chip/gadgetを組んで回路化。PLONK(KZG=準備1回/IPA=不要)。Poseidon有・EdDSA自作寄り

■ メリット

- lookup/custom gateなど高度で効率的な回路
- 回路別セットアップ不要
- Moproでモバイル対応

■ デメリット

- 学習コストが高く実装が難しい
- Baby JubJubのEdDSA実装は手間大
- Groth16ほど検証は安くない

■ 本案件 = 特殊機能が要らなければ費用対効果が悪い。積極採用しない

⑤ Plonky2 / Plonky3

■ できること = Rustで記述、STARK(FRI)。PoseidonはGoldilocksネイティブ高速・再帰が速い

■ メリット

- トラステッドセットアップ不要
- 再帰が非常に速く集約を軽くできる
- 証明生成が速い

■ デメリット

- Goldilocks体で現行EdDSA(BN254系)と不一致→非ネイティブ演算で高コスト
- 証明が大きくEVM検証はwrapが必要で高価
- 低レベルで記述が重い

■ 本案件 = 署名を含む端末回路には不向き。集約用途としては魅力

⑥ Nova / Sonobe（Folding）

■ できること = 計算を1ステップ回路に書きN個を逐次fold→最終1証明。arkworksでBN254/BabyJubJub

■ メリット

- 省メモリで端末やコーディネータ端末で集約できる
- Folding自体は回路別セットアップ不要
- 「N端末を1つに」に そのまま合う

■ デメリット

- ステップ回路への再設計が必要・事例が少ない
- 最終圧縮の証明系でセットアップが要る場合あり

■ 本案件 = 集約(M2)の代替として最有力の研究的選択肢

⑦ RISC Zero / SP1（zkVM）

■ できること = 普通のRustコードの実行の正しさをSTARKで証明。回路を意識しない

■ メリット

- 回路設計が要らず複雑なロジックを普通のコードで
- セットアップ不要・汎用性が非常に高い

■ デメリット

- 端末での証明生成が重い(今はサーバ/GPU前提)
- 証明が大きくEVM検証は圧縮が必要

■ 本案件 = 将来枠。今のモバイル端末証明には重い

⑧ Cairo / Stwo（Starknet）

■ できること = Cairoで記述、STARK。Poseidonネイティブ。モバイル大規模実証あり(5秒以内)

■ メリット

- セットアップ不要・モバイル証明の実績が豊富
- Poseidonが速い

■ デメリット

- 署名はStark曲線がnative・Baby JubJubは自作寄り
- 検証がStarknet前提でEVM+gnarkとエコシステムがずれる

■ 本案件 = 決済/集約レイヤをStarknetに替える話。EVM前提と不整合、見送り寄り

⑨ Mopro（モバイル配布ツールキット）

■ できること = 証明系でなく、circom/halo2/Noir等をモバイルアプリのバインディングに変換する橋渡し

■ メリット

- 現行circom回路をそのままモバイル化＝移行コスト最小
- Flutterパッケージがあり今のアプリと直結
- 後からNoir/halo2へ差し替えも同じツール

■ デメリット

- 証明系の性質(セットアップ・gas)は中身次第
- iOSシミュレータ非対応・Androidはarm64

■ 本案件 = 端末証明を今すぐ堅くする最短路

その他・新顔（参考 — 主軸は変えない）

■ モバイルで一応検討価値がある新顔
- Ligero/Ligetron = 低メモリ・constrained device向け設計
- o1js(Mina) = TypeScriptでZK・準備不要・再帰。ただし検証はMina前提

■ 集約(層3)を攻めるなら
- ProtoStar/ProtoGalaxy/HyperNova = Folding改良、Spartan = 準備不要SNARK

■ zkVM他 = Jolt / Miden / Triton / Nexus / Powdr / zkWASM

■ プローバ/lib = Expander(GKR最速級) / Binius(二進体) / Winterfell 他

■ 言語系 = Leo(Aleo) / zkLLVM・Placeholder(=nil;)

■ 結論: 主軸(Mopro＋circom/Noir・集約gnark/folding・EVM前提)は不変

目的別のおすすめ（結論）

やりたいこと	フレームワーク
今のまま端末証明を堅く・モバイル最適化	Mopro（circom+rapidsnark, Flutter）
★1トラステッドセットアップを根治	Noir + Barretenberg（gas増は覚悟）
多端末集約を端末側・省メモリで	Nova / Sonobe
集約の再帰を速く・準備不要に	Plonky2（or 現行 gnark 継続）
複雑ロジックを普通のコードで（将来）	RISC Zero / SP1

■ 総合の現実解 = 端末はMopro、集約はgnark継続 or Novaで端末寄せ、★1消すならNoir段階導入

モバイル実行の方法

各フレームワークを実機（Flutter）で動かすには

前提 — 「モバイルでZKP」とは何をすることか

```
端末でやること ： 入力 → witness生成 → proof生成　　（重い＝本題）
端末外でやること: verify（軽い・普通はチェーン/サーバ、端末でも可）
```

■ モバイル化 = 証明生成器(prover)を端末で動くバイナリにして呼ぶこと

- ネイティブ = arm64にクロスコンパイル（速い・本命）
- WASM = WebAssembly（移植は楽だがネイティブ比3.5〜4倍遅い）

■ 共通で効く制約

- RAMが支配的（3GB以上で安定・FibRace実測）
- iOSシミュレータは基本不可・Androidはarm64
- witness生成とproof生成は別物（witnesscalc等で軽くできる）

Flutter への繋ぎ方（3パターン）

方法	中身	向き
Mopro の Flutter パッケージ	prover を Swift/Kotlin/Dart バインディングに自動生成	最短・推奨
自前 FFI（dart:ffi）	arm64 の .so/.a を自分で作り Dart から呼ぶ	細かく制御したい時
MethodChannel プラグイン	ネイティブ側に実装し channel 越しに呼ぶ	既存ネイティブ実装がある時

■ 現行アプリは rapidsnark を端末で動かす構成（③に近い）

■ Mopro に寄せると①で標準化できる（実機ビルドまで検証済 → 08）

まとめ表（モバイル実行の観点）

FW	端末証明	モバイル化の方法	Flutter連携	一言
circom+rapidsnark	◎	元からarm64ネイティブ	Mopro / FFI	今すぐ動く
Noir+bb	◎	bbをarm64ネイティブ	Mopro(Flutter)	準備不要・速い
halo2	○	Rust→arm64	Mopro	回路作りが重い
Plonky2/3	△	Rust→arm64(FFI)	自前FFI	体不一致で不向き
Nova/Sonobe	◎(省メモリ)	Rust→arm64(FFI)	自前FFI	端末側集約向き
RISC Zero/SP1	×	実質リモートprover	API呼び出し	端末単独は不可
Cairo/Stwo	◎	Starknet SDK	別SDK	土台がStarknet
gnark	×	gomobile(重い)	—	集約サーバ専用
Mopro	—	上記を包む	◎公式Flutter	配布を一本化

モバイル実装の難易度（分岐点＝Mopro）

FW	難易度	理由
circom + rapidsnark	★☆☆ 易	元からarm64・Mopro公式Flutter対応
Noir + bb	★☆☆ 易	bbにモバイルビルド・Mopro公式Flutter対応
Cairo / Stwo	★★☆ 中	実証は豊富だがSDKがStarknet系＝土台乗り換え
halo2	★★☆ 中	Rust→arm64は可だが回路実装(chip)が難しい
Plonky2 / Nova	★★★ やや難	Flutter成熟プラグイン無く自前FFI
gnark / RISC Zero	★★★ 難	端末証明に不向き（サーバ/リモート前提）

■ 公式対応は circom / halo2 / Noir の3つ。現行もNoirも「易」＝モバイル化リスクは低い

あなたの Flutter アプリへの当てはめ（結論）

■ 今の circom 回路をモバイルで堅くする
- → Mopro の circom アダプタ＋mopro_flutter_package（移行最小）

■ ★1セットアップを消しつつ端末で速く
- → Noir+bb（Mopro Flutter経由）、回路をNoirへ移植

■ 複数端末を端末側で1つに畳む実験
- → Nova/Sonobe を FFI で（省メモリが効く）

■ 集約はサーバのままなら → gnark 継続（モバイル化しない）

■ 共通指針 = 端末証明はMoproで一本化・証明系は段階的に差し替え・RAM 3GB未満は要注意

フレームワーク概要

それぞれ「何者か」入門 — 何者/核の仕組み/背景/使われ方

読み方（前提の言葉）

■ 回路(circuit) = 証明したい計算を、足し算・掛け算の制約の集まりに変換したもの

■ 証明系 = 回路から証明を作り検証する数学エンジン（Groth16 / PLONK / STARK …）

■ トラステッドセットアップ = 一部の証明系が要る“鍵作りの儀式”。漏れると偽造の恐れ

■ フレームワーク = おおむね「回路を書く言語 + 証明系エンジン」の組み合わせ

A. 回路を書いてSNARK証明（1）

■ circom（＋snarkjs） ＝ 回路記述の専用言語＋証明を扱うJSライブラリ
- iden3製。circomlibにPoseidon/EdDSA/比較。Tornado/Semaphore等・本案件の現行

■ rapidsnark ＝ circom回路の高速Groth16証明生成器（C++/アセンブリ）
- iden3製。snarkjsと同じ証明を桁違いに速く。サーバ・モバイル端末証明

■ gnark ＝ Goで書くフルスタックZKPライブラリ
- ConsenSys製。再帰・Solidity出力。ロールアップ・本案件の集約(M2)

A. 回路を書いてSNARK証明（2）

■ Noir（＋Barretenberg） ＝ Rust風の書きやすい高水準言語
- Aztec製。中間表現ACIR→bb(UltraHonk)。準備は1回きり。zk-email/パスポート系

■ halo2 = 表現力の高い回路が書けるRust証明系（PLONK系）
- 元Zcash→Ethereum財団PSE(KZG版)。chip/gadget・lookup。Zcash Orchard/Scroll

B. STARK・再帰・Folding（集約向き）

■ Plonky2 / Plonky3 = 準備不要で再帰が速いSTARK系（PLONK＋FRI）
- Polygon Zero製。Goldilocks体で高速。Polygon zkEVM

■ Nova / HyperNova（＋Sonobe） = Folding（畳み込み）集約方式
- Microsoft Research発。省メモリ。長い反復計算・端末側集約

■ Cairo（＋Stwo） = StarkWareのSTARK言語＋VM（Starknetの実行環境）
- 準備不要・Poseidon速い。モバイル実証(FibRace)。検証はStarknet前提

C. zkVM ／ D. モバイルへの橋渡し

■ RISC Zero / SP1 = 普通のプログラム(Rust)の実行そのものを証明するzkVM
- RISC-VをSTARKで証明。回路設計が要らない。証明は重くサーバ/GPU前提

■ Mopro = 既存の証明器をモバイルアプリに載せる“橋渡し”ツールキット
- Ethereum財団PSE主導。circom/halo2/Noirを iOS/Android/Flutter バインディング化

E. これらを支える部品（土台ライブラリ）

■ circomlib = circom用の標準回路集（Poseidon・EdDSA・比較）。本案件が多用

■ arkworks = RustのZK部品群（有限体・曲線・Groth16）。Nova/ark-circomの土台

■ Barretenberg = AztecのC++証明バックエンド。Noirを動かす本体・WASM版も

■ witnesscalc = circomのwitness生成を高速化するC++。モバイルで効く

一言まとめ

名前	種別	一言
circom + snarkjs	言語＋証明系(Groth16)	ZK回路記述の定番・実績豊富
rapidsnark	高速プローバ	circom回路を端末で速く証明
gnark	Go 証明系	集約・再帰に強いサーバ向け
Noir + Barretenberg	言語＋証明系(Honk)	書きやすい・準備不要
halo2	証明系(PLONK)	高度だが難しい
Plonky2/3	STARK	準備不要・再帰高速
Nova / Sonobe	Folding	省メモリで逐次集約
Cairo / Stwo	STARK言語＋VM	Starknetの土台
RISC Zero / SP1	zkVM	普通のコードで書ける(重い)
Mopro	モバイル橋渡し	既存証明器をアプリ化

決定ドキュメント（ADR）

手法とフレームワークの選定 — 結論と根拠を1枚に

week19 / 05_決定ドキュメント_ADR

決めたいこと（Context）

■ 目的 = 各端末でRSSI/距離/複雑計算を秘匿証明→多端末を1つに→チェーンで共有・検証

- 端末はFlutterアプリ。セキュリティが核（★1トラステッドセットアップ含む）

■ 現状 = circom+rapidsnark(端末) / gnark(集約M2) / EVMでGroth16検証

- M1(集計)・M2(再帰)・#3b(網羅性)は設計/実装済

■ 論点 = この現状をどこまで維持し何を変えるか

- (a)モバイル配布の標準化 (b)★1セットアップの根治 (c)集約の軽量化

評価軸（重み）

軸	重み	理由
現行資産の再利用（移行コスト小）	★★★	動いている circom/gnark を捨てない
セットアップの健全性（★1根治）	★★★	プロジェクトの核＝セキュリティ
オンチェーン検証コスト（gas）	★★☆	チェーン記録が前提
モバイル証明の現実性	★★★	端末で秒級・省メモリ
集約適性（多端末を1つに）	★★☆	本案件の新規性
実装難度・事例の多さ	★★☆	開発リソース

選択肢と評価（要約）

候補	再利用	★1根治	gas	集約	総評
現行維持(circom+gnark)	◎	×	◎	◎(M2)	基準。★1が残る
＋Moproでモバイル標準化	◎	×	◎	◎	低コストで底上げ
Noir+bbへ端末を移行	△	◎	×(2.4M)	要再設計	★1根治だがgas増
halo2	△	○	△	重	費用対効果低
Plonky2で端末回路	×	◎	△	◎	署名と体不一致で不向き
Nova/Sonobeで集約	○	○	圧縮次第	◎	端末側集約に最適
Cairo/Stwo	×	◎	Starknet	○	EVM前提と不整合
RISC Zero/SP1	×	◎	△	○	端末に重い(将来)

決定 — 段階的アプローチ

■ フェーズ1（すぐ・低リスク）= Moproでモバイル標準化

・ 現行circom+rapidsnarkのままmopro_flutter_packageに載せ替え。回路・検証・集約は変えない

■ フェーズ2（並行・研究）= 集約の端末寄せをNova/Foldingで検証

・ per-witness回路はそのまま、集約だけNovaでPoC。gnarkはフォールバックで残す

■ フェーズ3（条件付き）= ★1根治のためNoir評価

・ 端末回路をNoirへ移植して比較。ただしgas増(2.4M)と集約作り直しを許容できる場合のみ

明示的に採らない（Rejected）

■ halo2 = 回路実装(chip/gadget)の難度が高く、必要機能に対し費用対効果が低い

■ Plonky2を端末回路に = EdDSA(BabyJubJub)とGoldilocks体が不一致で非ネイティブ演算が重い

- （集約用途としては将来検討可）

■ Cairo/Stwo = モバイル証明は優秀だが検証がStarknet前提でEVM+gnarkと不整合

■ RISC Zero/SP1 = 端末単独の証明生成が重く、当面リモートprover前提でオンデバイスにならない

結果として起きること（Consequences）

■ 良い点

・ 現行資産を保ったままモバイルが堅く(F1)、集約の軽量化に道筋(F2)、★1根治の判断材料(F3)

・ どの段階でも「動く状態」を保てる

■ コスト/リスク

・ Moproのビルド制約(iOSシミュ不可/arm64)、Novaは事例少なく自作要素、Noirはgas増と集約再設計

■ 未決定の前提

・ 実機の証明時間・メモリは未実測(→06)、信頼モデルの差(→07)

一言結論

現行(circom+rapidsnark+gnark)を土台に、フェーズ1でMoproによる
モバイル標準化を即実施。集約はNova/Foldingを研究的に検証し、
★1根治が必要になったらNoirをgasトレードオフ込みで評価する。

■ 手法（層2エンジン・層3集約）を先に固め、フレームワークはそれを実装する乗り物として選ぶ

ハンズオン結果

実測で circom と Noir を比較（proof→verifyまで）

現行 circom 回路の実測

回路	制約数	証明生成	検証
uav_witness_attestation（フル）	11,785	≈1.4s(snarkjs)	OK(≈348k gas)
remoteid_consistency_rel（RSSI単体）	649	—	—

■ 訂正 = 従来「~18k制約」は概算、実測は 11,785（≈12k）。Groth16 では小規模

■ 証明生成は snarkjs で ≈1.4s。rapidsnark(arm64) ならさらに速い＝端末も実用域

Noir 版の実測（proof→verify まで）

■ 環境 = nargo 1.0.0-beta.22 / bb 5.0.0-nightly（UltraHonk）

コマンド	結果
nargo test	3/3 通過（正常 count=2 / 合意未達 fail / 束縛違反 fail）
nargo info	ACIR 92 opcodes
nargo execute	witness 生成成功・回路出力 = 2（期待通り）
bb gates	回路サイズ 2,953 ゲート
bb prove	証明生成 0.11s・proof ≈14.6 KB
bb verify	「Proof verified successfully」・0.02s

circom vs Noir（暫定比較・同条件でない）

項目	circom（Groth16）	Noir（UltraHonk）
Poseidon 1回（同機能・公平）	415制約	404ゲート ＝ ほぼ同等
EdDSA署名検証1回	8,086制約（実測）	未実測 / 文献≈24,472
証明生成 ※規模が違う	≈1.4s（フル11.8k）	0.11s（小PoC・署名なし）
証明サイズ	定数・小（~200B級）	≈14.6 KB（大）
セットアップ	回路別セレモニー要	不要（universal）
オンチェーン検証	≈348k gas	≈2.4M gas（文献）

EdDSA と Noir 暗号ライブラリの成熟度

■ Poseidon は同機能で実測比較: circom 415 ≒ Noir 404（ほぼ同等）

- → ハッシュのコストは近く、Noir移行で回路が極端に重くはならない見込み

■ EdDSA は circom 側を実測（8,086制約）、Noir 側は実測に至らず

■ つまずき = Noir公式 eddsa/ec の最新タグが u1（beta.22で削除）を使いコンパイル不可

- ec の main は修正済みだが未タグ、beta.22 は git依存に tag 必須で参照できず

■ 知見 = 署名検証のような重い回路は Noir だとライブラリのバージョン整合で詰まる

- circomlib は枯れていて即使える = この成熟度の差自体が移行コスト

読み取り（結論）

■ 現行回路は小さい（≈12k制約）＝証明が回るか自体は問題にならない
  ・ デスクトップ1.4s / 端末も rapidsnark で実用域 / FibRace一般傾向 <5s

■ 差が出るのは制約数より セットアップ・オンチェーンgas・集約方式

■ Noir に替えても証明生成は速い一方、オンチェーン検証gasは約7倍

■ トレードオフ = Noir(準備不要・端末で速い) / Groth16(小さく安いオンチェーン検証)

導入のつまずき ／ 次の一手

■ 導入のつまずき（記録）

- nargo beta.23 はバイナリ未添付 → beta.22 で回避
- mopro 導入に rustc 1.91+ 必要 → rustup update（1.90→1.97）
- Noir の Android は Zig 必須（barretenberg ABI）→ Android は circom が楽
- Flutterテンプレートが mopro_example_app 前提 → import名を修正

■ 次の一手

- 本物の回路(uav_witness_attestation.zkey)に差し替えて実機測定
- Poseidon/EdDSA を Noir版に足し、同機能での完全比較を完成（EdDSAはlib vendoring）

信頼モデル比較

誰がズルできて、何が止めるか

信頼が必要になる「4つの場所」

① セットアップ … 鍵作りの儀式は健全か（漏れると証明を偽造できる）
② プローバ … 証明を作る人は生データを見るか（プライバシー・独占）
③ 集約者 … まとめる人は結果を歪められるか（改ざん・検閲）
④ 入力 … 入れたRSSI/GPS/端末は本物か（真正性・Sybil）

■ ①②③は暗号(ZKP)で守れる。④はZKPの外（アテステーション等）で裏取り

■ 方針 = 改ざんは暗号で不可／検閲は網羅性or合意／入力は真正性／鍵は正規セレモニー

① セットアップの信頼（証明系ごとに違う）

証明系	セットアップ	信頼の中身	綻びると	本案件
Groth16（現行）	回路ごとに儀式	1人でも正直なら安全(1-of-N)	全員結託で偽造	★1・実施済
PLONK/Honk	1回のSRS	同上(1-of-N)・回路別不要	同上	★1を回路別から解放
STARK/Folding	不要	信頼点なし(transparent)	—	★1が構造的に消える

■ 現行の弱点 = 回路を変えるたびに儀式。Noirで頻度減、STARK/Foldingで儀式が消える

② プローバの信頼（集約手法で大きく変わる）

集約手法	誰が証明を作るか	生データを見るのは	壁
オンチェーン集計(K-of-N)	各端末が自分の分	各端末＝自分の分のみ	◎分散
回路内バッチ(1回路にN端末)	単一プローバ	その1人が全端末分	×独占
再帰的合成(recursion)	各端末＋集約者	集約者は証明のみ	◎
Folding	各端末＋畳み込み役	畳み込み役は証明のみ	○

■ 重要 = 回路内バッチは「1人が全座標を持つ」→分散の趣旨と矛盾。実運用は再帰 orFolding

③ 集約者の信頼（改ざんと検閲を分ける）

攻撃	集約者にできるか	何が止めるか	本案件
改ざん（結果を偽る）	✕ できない	ZKPの健全性(偽証明は落ちる)	全証明系で担保
検閲（有効票を間引く）	○ できてしまう	網羅性証明 or BFT合意＋全複製	#3b 設計済
なりすまし集約(空スロット)	検証すれば不可	集約時に各proofも検証	M2 が担保

■ 改ざんは無理だが検閲は仕組みで別途防ぐ。証明系を替えても構図は同じ

④ 入力の真正性（ZKPの外・全手法共通）

リスク	内容	対策	本案件
端末なりすまし	偽端末が証明を投稿	端末アテステーション(TEE)	★2 / 設計済
Sybil	1人が多数端末を装う	登録メンバーシップ＋1端末1票	#3a 実装済
入力捏造	偽のRSSI/GPSを入力	ドローン署名・freshness・相互整合	★2 / 回路統合済
較正すり替え	A/nを意図的にずらす	較正手順の固定・監査	実機較正で対処

■ どのフレームワークを選んでも④は解決しない。ZKP外の仕組みで裏取り（概ね実装/設計済）

まとめ — フレームワーク横断

観点	変わるか	効いてくるもの
① セットアップの信頼	証明系で変わる	Groth16=回路別 / Noir=universal / STARK・Folding=不要
② プローバの信頼	集約手法で変わる	回路内バッチは避ける / 再帰・Foldingは分散◎
③ 集約者の検閲	変わらない	網羅性(#3b) or 合意という別レイヤが常に必要
④ 入力の真正性	変わらない	アテステーション/登録/相互整合（ZKP外）

■ 結論 = 集約は再帰/Folding(分散)を維持、証明系は必要ならNoir/STARKで①を下げ、

■ ③検閲・④入力は既存の #3b・アテステーションで守る

Mopro モバイル PoC 結果

circom プローバを Flutter/Android に載せる（APKまで）

結論 — 実機ビルドは確認済み（証明は次段）

■ circom(Groth16)プローバをMopro経由でFlutterに載せ、Android(arm64)の

■ インストール可能なAPKまでビルドできた（＝“アプリまで作れた”段階）

- ＝フェーズ1(Moproでモバイル標準化)が実機ビルドで裏取り。オンデバイス証明は次段

■ 生成物（実測）

- libmopro_uav_circom.so（arm64ネイティブ証明器・3.0MB）
- mopro_flutter_bindings（iOS+AndroidのDart/Kotlin/Swiftバインディング）
- Flutterアプリ（generateCircomProof / verifyCircomProof を呼ぶ）
- app-debug.apk（53MB, arm64）← 端末にインストール可能

■ 残る手動ステップ＝実機を接続して flutter run（オンデバイスで証明→検証を目視）

実施フロー（再現手順）

```
                                    # 1. ツール
rustup update stable                 # mopro依存が rustc 1.91+ → 1.90→1.97
cargo install mopro-cli --locked     # mopro CLI 0.3.7
rustup target add aarch64-linux-android aarch64-apple-ios ...

# 2. プロジェクト生成（circomアダプタ）
mopro init --adapter circom --project-name mopro_uav_circom

# 3-4. Android/Flutter バインディング＋アプリ雛形
mopro build --platforms android --architectures aarch64-linux-android
mopro build --platforms flutter --architectures aarch64-apple-ios ...
mopro create --framework flutter

# 5. APKビルド（arm64のみに絞ると速い）
flutter build apk --debug --target-platform android-arm64
  → app-debug.apk (53MB)
```

つまずき（重要な記録）

事象	原因	回避
nargo が tar 展開エラー	beta.23 がバイナリ未添付	noirup -v 1.0.0-beta.22
cargo install mopro 失敗	依存が rustc 1.91+ 要求	rustup update stable(→1.97)
Noir の Android ビルド失敗	barretenberg が Zig の libc++要求	Android は circom で回避
cargo expand エラー	flutter_rust_bridge が依存	cargo install cargo-expand
APK が Dart コンパイル失敗	template の import名不一致	mopro_example_app→実名に置換
APK ビルドが時間切れ	4 ABI 分の Rust を毎回ビルド	--target-platform android-arm64

教訓 ／ 次の一手

■ 教訓

・ AndroidでNoirを使うならZigが要る。circomはarkworks系でZig不要＝Androidはcircomが楽

・ MoproのFlutterテンプレートはプロジェクト名 mopro_example_app 前提、別名init時は手直し

■ 次の一手

・ 本物の回路(uav_witness_attestation.zkey)に差し替えて同手順でビルド

・ 実機で flutter run、オンデバイスで証明生成→検証の時間を実測

・ iOS もビルドするなら署名設定(開発者証明書)を Xcode で

■ 意義 = フェーズ1(Mopro標準化)を「設計上の推奨」→「実機ビルドで裏取り済み」に格上げ